

11. DAO Pattern và Data Access trong Spring Boot

11.1. Giới thiệu về DAO Pattern

DAO (Data Access Object) Pattern là một mẫu thiết kế (design pattern) được sử dụng để tách biệt logic truy cập dữ liệu khỏi logic nghiệp vụ của ứng dụng. Mục tiêu chính của DAO Pattern là cung cấp một lớp trừu tượng (abstraction layer) trên tầng persistence (lưu trữ dữ liệu), cho phép ứng dụng tương tác với các nguồn dữ liệu khác nhau (như cơ sở dữ liệu, file, dịch vụ web) mà không cần biết chi tiết về cách thức truy cập dữ liệu đó.

11.1.1. Lợi ích của DAO Pattern

- **Tách biệt mối quan tâm (Separation of Concerns):** Logic truy cập dữ liệu được tách biệt rõ ràng khỏi logic nghiệp vụ, giúp mã nguồn dễ đọc, dễ hiểu và dễ bảo trì hơn.
- **Khả năng thay đổi nguồn dữ liệu:** Nếu bạn cần thay đổi cơ sở dữ liệu (ví dụ: từ MySQL sang PostgreSQL) hoặc công nghệ persistence (ví dụ: từ JPA sang JDBC), bạn chỉ cần thay đổi lớp DAO mà không ảnh hưởng đến tầng nghiệp vụ.
- **Dễ dàng kiểm thử (Testability):** Các lớp DAO có thể được mock (giả lập) dễ dàng trong quá trình kiểm thử đơn vị (unit testing) của tầng nghiệp vụ.
- **Tái sử dụng:** Các phương thức truy cập dữ liệu có thể được tái sử dụng trong nhiều phần khác nhau của ứng dụng.

11.2. Triển khai DAO Pattern trong Spring Boot

Trong Spring Boot, DAO Pattern thường được triển khai thông qua Spring Data JPA (hoặc Spring Data JDBC, Spring Data MongoDB, v.v.). Spring Data JPA cung cấp một cách tiếp cận rất hiệu quả để tạo các lớp Repository (thường được gọi là DAO trong ngữ cảnh này) mà không cần viết nhiều mã nguồn.

11.2.1. Spring Data JPA Repository

Spring Data JPA giúp đơn giản hóa việc triển khai tầng truy cập dữ liệu bằng cách tự động tạo các triển khai cho các interface repository. Bạn chỉ cần định nghĩa một interface mở rộng `JpaRepository` (hoặc `CrudRepository` , `PagingAndSortingRepository`).

Ví dụ:

Entity `Product` :

Java

```
import javax.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private String category;

    // Constructors, Getters, Setters
    public Product() {}

    public Product(String name, String description, BigDecimal price, String
category) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.category = category;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description =
description; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public String getCategory() { return category; }
```

```
public void setCategory(String category) { this.category = category; }  
}
```

Product Repository (DAO):

Java

```
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
import java.util.List;  
  
@Repository  
public interface ProductRepository extends JpaRepository<Product, Long> {  
    // Spring Data JPA tự động tạo triển khai cho các phương thức CRUD cơ  
    bản:  
    // save(), findById(), findAll(), deleteById(), count(), etc.  
  
    // Bạn có thể định nghĩa các phương thức truy vấn tùy chỉnh theo quy ước  
    đặt tên:  
    List<Product> findByCategory(String category);  
    List<Product> findByNameContainingIgnoreCase(String name);  
    List<Product> findByPriceGreaterThan(BigDecimal price);  
}
```

Trong ví dụ trên, `ProductRepository` là interface DAO của chúng ta. Spring Data JPA sẽ tự động tạo một triển khai cho interface này tại thời điểm chạy. Bạn không cần phải viết bất kỳ mã nguồn nào cho các phương thức CRUD cơ bản.

11.2.2. Sử dụng `@Query` cho các truy vấn phức tạp

Khi các phương thức truy vấn theo quy ước đặt tên không đủ, bạn có thể sử dụng annotation `@Query` để viết các truy vấn JPQL (Java Persistence Query Language) hoặc Native SQL.

Java

```
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.data.jpa.repository.Query;  
import org.springframework.data.repository.query.Param;  
import org.springframework.stereotype.Repository;  
import java.math.BigDecimal;  
import java.util.List;  
  
@Repository
```

```

public interface ProductRepository extends JpaRepository<Product, Long> {

    // JPQL Query
    @Query("SELECT p FROM Product p WHERE p.category = :category AND p.price > :minPrice ORDER BY p.name ASC")
    List<Product> findProductsByCategoryAndMinPrice(@Param("category") String category, @Param("minPrice") BigDecimal minPrice);

    // Native SQL Query
    @Query(value = "SELECT * FROM products WHERE available = TRUE ORDER BY created_at DESC LIMIT 10", nativeQuery = true)
    List<Product> findTop10AvailableProducts();
}

```

11.2.3. Tích hợp với Service Layer

Các Repository (DAO) thường được inject vào tầng Service để thực hiện các logic nghiệp vụ.

Java

```

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Transactional(readOnly = true)
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    @Transactional(readOnly = true)
    public Optional<Product> getProductById(Long id) {
        return productRepository.findById(id);
    }

    @Transactional

```

```

public Product saveProduct(Product product) {
    // Thêm logic nghiệp vụ nếu cần trước khi lưu
    return productRepository.save(product);
}

@Transactional
public void deleteProduct(Long id) {
    productRepository.deleteById(id);
}

@Transactional(readonly = true)
public List<Product> getProductsByCategory(String category) {
    return productRepository.findByCategory(category);
}

@Transactional
public Product updateProductPrice(Long id, BigDecimal newPrice) {
    return productRepository.findById(id).map(product -> {
        product.setPrice(newPrice);
        return productRepository.save(product);
    }).orElseThrow(() -> new RuntimeException("Product not found"));
}
}

```

11.3. Data Access Object (DAO) truyền thống (JDBC Template)

Trong một số trường hợp, đặc biệt khi bạn không muốn sử dụng JPA/Hibernate hoặc cần kiểm soát chi tiết hơn về SQL, bạn có thể sử dụng Spring JDBC Template để triển khai DAO.

11.3.1. Cấu hình JDBC Template

Java

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import javax.sql.DataSource;

@Configuration
public class JdbcConfig {

    @Bean

```

```
    public JdbcTemplate jdbcTemplate(dataSource) {  
        return new JdbcTemplate(dataSource);  
    }  
}
```

11.3.2. Triển khai DAO với JDBC Template

Java

```
import org.springframework.jdbc.core.JdbcTemplate;  
import org.springframework.stereotype.Repository;  
import java.util.List;  
import java.util.Optional;  
  
@Repository  
public class ProductJdbcDao {  
  
    private final JdbcTemplate jdbcTemplate;  
  
    public ProductJdbcDao(JdbcTemplate jdbcTemplate) {  
        this.jdbcTemplate = jdbcTemplate;  
    }  
  
    public List<Product> findAll() {  
        String sql = "SELECT id, name, description, price, category FROM  
products";  
        return jdbcTemplate.query(sql, (rs, rowNum) -> {  
            Product product = new Product();  
            product.setId(rs.getLong("id"));  
            product.setName(rs.getString("name"));  
            product.setDescription(rs.getString("description"));  
            product.setPrice(rs.getBigDecimal("price"));  
            product.setCategory(rs.getString("category"));  
            return product;  
        });  
    }  
  
    public Optional<Product> findById(Long id) {  
        String sql = "SELECT id, name, description, price, category FROM  
products WHERE id = ?";  
        return jdbcTemplate.queryForObject(sql, new Object[]{id}, (rs,  
rowNum) -> {  
            Product product = new Product();  
            product.setId(rs.getLong("id"));  
            product.setName(rs.getString("name"));  
            product.setDescription(rs.getString("description"));  
        });  
    }  
}
```

```

        product.setPrice(rs.getBigDecimal("price"));
        product.setCategory(rs.getString("category"));
        return product;
    });
}

public int save(Product product) {
    String sql = "INSERT INTO products (name, description, price,
category) VALUES (?, ?, ?, ?)";
    return jdbcTemplate.update(sql, product.getName(),
product.getDescription(), product.getPrice(), product.getCategory());
}

public int update(Product product) {
    String sql = "UPDATE products SET name = ?, description = ?, price =
?, category = ? WHERE id = ?";
    return jdbcTemplate.update(sql, product.getName(),
product.getDescription(), product.getPrice(), product.getCategory(),
product.getId());
}

public int deleteById(Long id) {
    String sql = "DELETE FROM products WHERE id = ?";
    return jdbcTemplate.update(sql, id);
}
}

```

11.4. Bản chất của DAO Pattern và Data Access trong Spring Boot

Bản chất của DAO Pattern và Data Access trong Spring Boot là cung cấp một cách tiếp cận có cấu trúc và hiệu quả để quản lý việc tương tác với tầng lưu trữ dữ liệu. Spring Boot, đặc biệt thông qua Spring Data JPA, đã đơn giản hóa đáng kể việc triển khai DAO Pattern bằng cách giảm thiểu boilerplate code và tự động tạo các triển khai repository. Điều này cho phép các nhà phát triển tập trung vào việc định nghĩa các interface truy vấn dữ liệu thay vì viết mã nguồn lặp đi lặp lại. Việc tách biệt tầng truy cập dữ liệu khỏi logic nghiệp vụ không chỉ cải thiện khả năng bảo trì và kiểm thử của ứng dụng mà còn giúp ứng dụng linh hoạt hơn khi cần thay đổi công nghệ persistence. Tóm lại, DAO Pattern trong Spring Boot giúp xây dựng các ứng dụng có kiến trúc sạch, dễ mở rộng và bền vững.